Artificial Life Simulations
Electronics-ICT

(B-KUL-JLI1QS)

Arne Vanmarcke, Thibault Dekock

May 14, 2025



Instructor: Thomas Feys

1 Introduction

Artificial life is een onderzoeksgebied dat zich richt op het simuleren van levende systemen aan de hand van computationele modellen. Interacties tussen individuele entiteiten ontstaan hier uit eenvoudige regels en interacties, in plaats van complexe, gedetailleerde scripts. In dit project onderzoeken we een van de bekendste modellen van artificial life: Boids.
Boids bootsen het gedrag na van groepen zoals zwermen vogels, scholen vissen of zelfs mensenmassa's.

2 Model details

De complexiteit van boids ontstaat door de interactie van individuele agenten (de boids, in dit geval) die zich houden aan een set van eenvoudige regels. Een eenvoudige simulatie kan gebruik maken van de volgende regels:

- **Separatie**: vermijd botsingen met nabije kuddegenoten.
- **Afstemming**: beweeg in dezelfde richting als nabije kuddegenoten.
- **Cohesie**: beweeg naar de gemiddelde positie (massamiddelpunt) van lokale vlokgenoten.

Uit enkel deze lokale interacties ontstaat spontaan globaal groepsgedrag, zonder centrale besturing. Natuurlijk kunnen er nog complexere regels worden toegevoegd, zoals het vermijden van obstakels (effect van angst), overbrengen van emoties (met feromonen), leiderschap en het zoeken naar een doel. In dit project, voegen we de volgende regels toe:

- **Zoek naar doel**: wanneer de linker-muisknop op het scherm wordt ingedrukt, wordt een doel toegevoegd en bewegen de boids naar die locatie. De rechter-muisknop verwijdert dit doel.
- **Verlaat vlok**: wanneer we op de spatie drukken, bewegen de boids weg van elkaar, waarna ze nieuwe vlokgenoten zoeken.
- **Verkleur**: elke boid wordt voorgesteld door een pixel waarvan zijn kleur verandert op basis van de absolute snelheid.

3 Algemene structuur van de simulatie

In onze simulatie worden boids gemodelleerd als individuele agenten met een positie, snelheid, versnelling en een kleur die hun snelheid visualiseert. Elke boid past per frame verschillende gedragsregels toe, zoals beschreven in Sectie 2.
Voor zowel de CPU- als GPU-implementatie delen we de volgende componenten:

- `Boid.h` – bevat de structuur en basisbewerkingen van een boid.
- `Vector.h` – bevat vectoroperaties zoals optellen, schalen, normaliseren, en afstandsberekeningen.
- `Utils.h` – bevat kleurinterpolatie, logging, en visuele hulpmiddelen.

De boids worden initieel willekeurig geplaatst met een snelheid in een willekeurige richting. Per frame worden de boids geüpdatet op basis van:

a. Buurdetectie (maximaal 128 buren binnen een straal van 35 pixels)

b. Berekening van gedrag (alignment, cohesion, separation, attractie of explosie)

c. Beweging op basis van de resulterende krachten

Visualisatie
De simulatie gebruikt SDL3 om boids weer te geven als cirkeltjes. Elke boid krijgt een kleur op basis van zijn snelheid: blauw (traag) tot rood (snel), via een kleurverloop. Ook het doel wordt weergegeven als een grotere rode stip.

Interacties

- **Linker muisklik**: voegt een doel toe waarheen boids zich verplaatsen.
- **Rechter muisklik**: verwijdert het actieve doel.
- **Spatiebalk**: activeert een 'explosie' waarbij boids tijdelijk hun groep verlaten.

Logging
Tijdens de simulatie worden het aantal frames en de framerate per seconde opgeslagen. Deze data wordt aan het einde weggeschreven naar een JSON-bestand `output_GPU (of CPU).json`, zodat prestaties later geanalyseerd kunnen worden.

4 CPU-implementatie

De CPU-versie is een sequentiële implementatie in C++. Elke boid wordt per frame apart verwerkt in een enkele `for`-lus.

Nabuurdetectie
Voor elke boid worden buren gezocht via brute-force. Dit is computationeel duur: voor n boids zijn er $O(n^2)$ vergelijkingen per frame, tenzij het aantal buren vroegtijdig beperkt wordt tot 128.

Gedrag
De gedragsregels worden binnen de for loop per boid toegepast:

- **Afstemming**: gemiddelde snelheid van buren
- **Cohesie**: beweeg naar het massamiddelpunt van buren
- **Separatie**: repulsie van te nabije boids
- **Aantrekking**: richting het doel, indien aanwezig
- **Explosie**: (tijdelijke) kracht weg van het centrum van de buren

5 GPU-implementatie

Wanneer de transformatie naar CPU- naar GPU-code wordt gemaakt, worden de boid en Vector2 klasse methoden gemarkeerd met *__device__* zodat iedere methode kan gebruikt worden wanneer de Vector of Boid op de GPU geladen wordt. Ook worden GPU specifieke functies aangeduid met een *__device__* en worden kernel functies aangemaakt. Een voorbeeld hiervan is de *initBoidsKernel* kernel, waarbij boids parallel worden geïnitialiseerd.
Een tweede belangrijk element bij het overschakelen naar CUDA: het random genereren van getallen via **curand**. Bij de CPU versie wordt de standaard srand gebruikt, die enkel werkt op host functies. Hierdoor kunnen deze niet gebruikt worden voor GPU berekeningen. `curand` is de CUDA-versie van `srand`. Hierbij wordt op voorhand een array van staten geïnitialiseerd, die vervolgens gebruikt worden om willekeurige getallen te genereren op de GPU.

Initialisatie
Boids worden willekeurig geïnitialiseerd met `curand` en opgeslagen in device-memory. De snelheden bepalen hun kleur via interpolatie.

Simulatiekernel
De kern van de update gebeurt in `SimulateBoidKernel`, waarbij elk CUDA-block een reeks boids verwerkt. Nabuurdetectie, gedragsregels en positieberekening verlopen parallel.

Explosiegedrag
Als de spatiebalk wordt ingedrukt, wordt een explosiekracht toegepast. Een extra willekeurige richting wordt toegevoegd met `curand`, zodat boids uiteenstuiven.

Synchronisatie
Na elke frame worden de boids teruggekopieerd naar de cpu voor weergave via SDL.

6 Vergelijking en analyse

Zoals te zien in figuur 1 en tabel 1, behaalt de CPU een aanzienelijk lagere fps dan de GPU. De fps start duidelijk hoger in het begin, waarna de deze sterk daalt. Naar het einde van het programma toe stabiliseert het terug. Deze daling in FPS komt door het groeperen van de boids naar een grote groep. Elke boid moet zo telkens meer neigbours vinden en moet meer berekeningen uitvoeren.

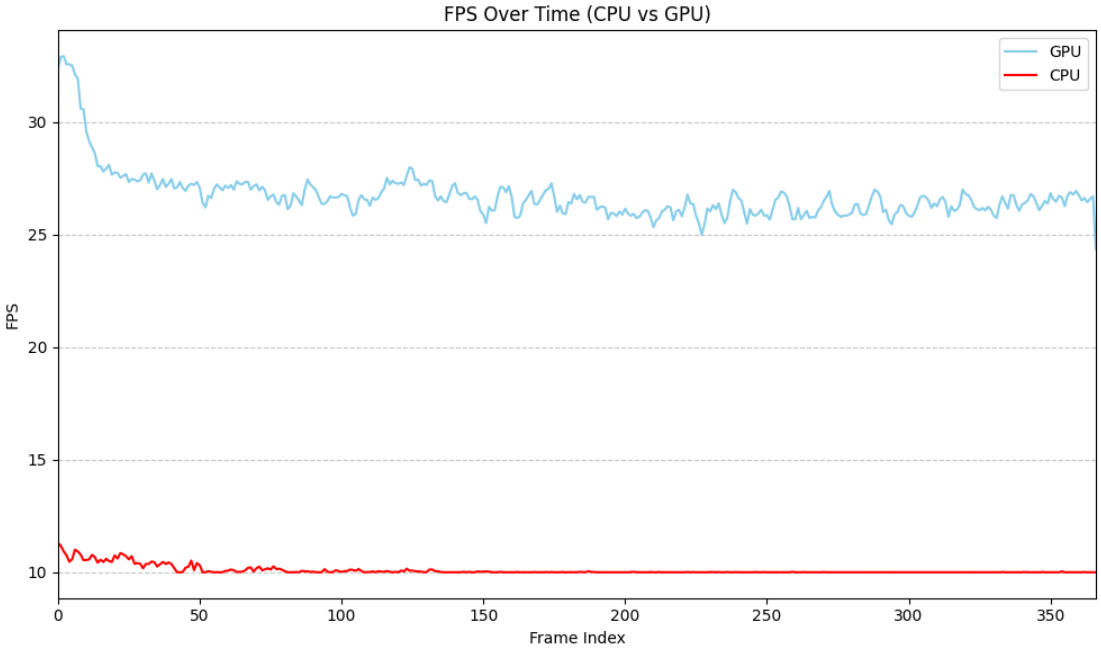


Figure 1: CPU and GPU fps over time

	CPU	GPU
frames	10k	10k
runtime (seconds)	1082.75	374.91
avg fps	10.03	26.73
min fps	10	24.37
max fps	11.3	32.93

Table 1: Vergelijking van fps statistieken

Een paar punten worden duidelijk bij het vergelijken van de CPU- en GPU-implementaties. Ten eerste is de gemiddelde fps-tijd van de CPU ongeveer 2,6 keer trager dan die van de GPU. Ten tweede gaat de GPU-implementatie beter om met schaalbaarheid, doordat deze in staat is om voor elke boid parallel naar naburige boids te zoeken. Dit is duidelijk te zien op Figuur 2 waar de runtime voor de CPU aanzienelijk stijgt bij een groeiend aantal boids.

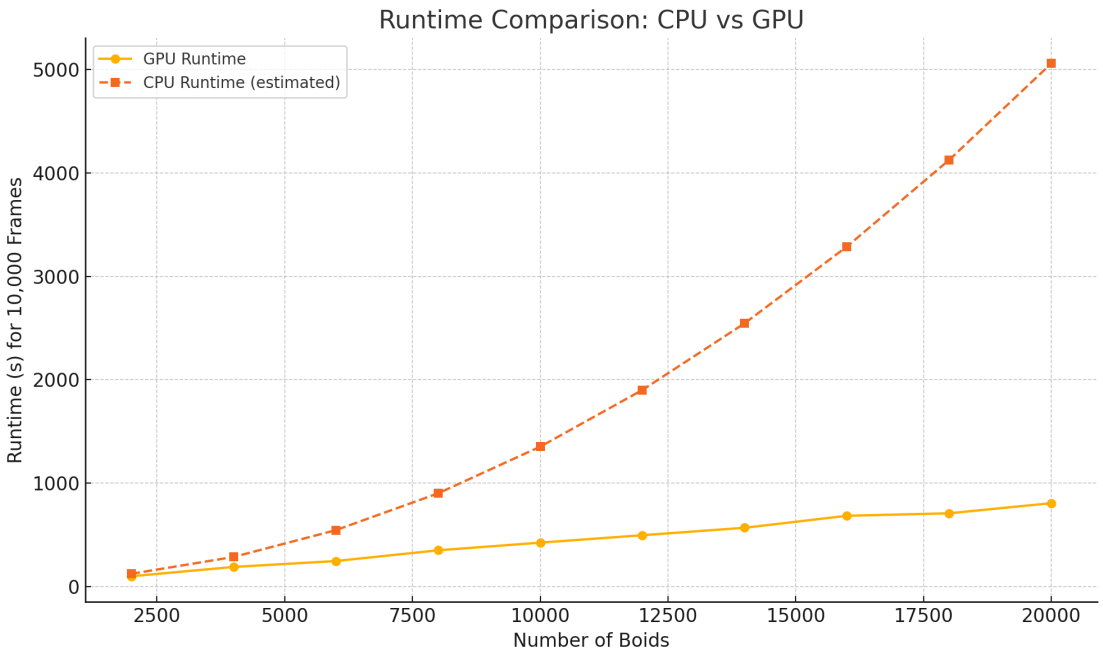


Figure 2: Runtime vergelijking CPU vs GPU

De code is terug te vinden op github (here).

De main branch bevat de GPU code, op de CPU branch is de CPU versie van de code terug te vinden.